Verification of the score-based donation game engine

13 October 2026

1 Summary

This report lists the checks made on the engine in `engine/`, the replicator-dynamics GUI in `gui/` and the scripts that now run on them. The repository state is `main` at commit `5194351`, the merge of pull request 1 (`https://github.com/bdotloh/donation-game-with-score-rep/pull/1`). The notation is that of the notes *Calculating the Stable Reputation Distribution in Infinite Population Models of Indirect Reciprocity*, which were read in full (pages 1–8).

Check	Against	Result
Binary stable scores, payoffs and rates	`helper_funcs.jl`, 400 cases	All 400 agree to 10^{-12}
Binary ESS	`helper_funcs.jl`, 3192 rows (809 ESS)	All 3192 rows agree
ESS sweep script	`helper_funcs.jl`, all 1296 CN-only norms, noise 0 and 0.01	No differences
Defectors against Discriminators	Notes, pages 5–6	Every number reproduced, including the 1.10×10^{-5} and 7.36×10^{-5} differences
ϕ-Disc	Notes, pages 7–8	Rates agree to 6×10^{-17}; $\pi_0 = 0.2555$, $\mathbb{E}[\text{score}] = 1.889$; gamma-function solution agrees at every score
Ternary classes and ϕ-Disc	Independent models (Section 5)	Agree to 2×10^{-15} (rates, chains); simulation within sampling noise
Replicator dynamics and rest points	Closed forms and games with known answers	All pass
Earlier scripts moved onto the engine	Old code and committed results	Agree to rounding, except where the old score cap bound
Portability	Python 3.9 / numpy 1.26 / scipy 1.11 (Rāpoi) and Python 3.13 / numpy 2.5 / scipy 1.18	Tests pass on both
Test suite	650 tests (2 browser tests run separately)	All pass

Two things are worth knowing before reading on. First, the notes contain no ternary section, so the ternary results have no closed form to be compared with; Section 5 describes how they were checked instead. Second, the notes print the ϕ-Disc norm on page 7 as `CN:000000-+`, which is not a valid norm (it would let a score-0 donor lose a score). The code `CN:00000-0+` is the one the engine uses and the one that reproduces the notes' numbers.

2 What was tested

The engine computes, for a well-mixed population of n strategies, the stable score distribution of each strategy as the stationary distribution of a non-linear birth–death chain (notes, Eqs. 1–6), with transition rates

$$g_j^{(i)} = (1-\epsilon)\, G_j^{(i)} + \epsilon\, \epsilon_\uparrow, \qquad f_j^{(i)} = (1-\epsilon)\, F_j^{(i)} + \epsilon\, \epsilon_\downarrow, \qquad \epsilon_\uparrow + \epsilon_\downarrow = 1.$$

Strategies are binary (score 0 or at least 1), ternary with a cut-off τ (score 0, 1 to τ, above τ) or ϕ-Disc (help a receiver with a positive score with probability $j^{-\phi}$, where j is the donor's score); every decision point takes a probability in $[0, 1]$. Scoring norms are Frean and Marsland

barcodes, binary (8 characters) or ternary (18 characters). Scores have no upper limit; the chains run until they converge or reach a maximum number of steps. On top of this the repository has the replicator dynamics (the score chains restart from the initial distribution at every step), the rest-point finder, the Dash GUI, an ESS sweep for Slurm, and the earlier scripts moved onto the engine.

3 Binary engine against the Julia code

The user's Julia code is the reference for everything binary. The script `make_reference.jl` runs `helper_funcs.jl` and saves its output as tab-separated files in `tests/reference/`. The `OrderedCollections` package, which was not available, is replaced by `Dict` for this.

Stable scores. `stable_cases.tsv` holds 200 one-strategy and 200 two-strategy cases, with noise 0 (203 cases), 0.01 (93) and 0.1 (104), and `maxn` in $\{0, 1, 7, 100, 1000\}$. For each case the engine's final score distributions, payoffs and the three rates $\alpha = g_1$, $\beta = g_j$, $\gamma = f_j$ are compared with Julia's, with an absolute tolerance of 10^{-12}. All 400 cases agree. Payoffs follow Julia's convention: for one strategy they are computed from the distribution before the last step, for two strategies from the final one.

ESS. `ess_sweep.tsv` holds the ESS that `find_ESS` finds among the 16 binary strategies for 1596 norms at noise 0 and at noise 0.01 (3192 rows, 809 ESS in total, with $\rho_A = 0.99$, `maxn` 100 and fitness threshold 10^{-5}). `tests/reference/compare_ess.py` compares every row and finds the same ESS lists in all 3192; the test suite checks a fixed random sample of 60 rows.

ESS sweep script. `scripts/ess_sweep.py`, which runs the same search for any norm set (and is what runs on Rāpoi), was run on all 1296 CN-only binary norms at noise 0 and 0.01 and gave the same ESS lists as Julia for every norm. The run–resume–merge cycle and the Julia agreement on a small set are part of the test suite.

4 The notes, pages 1–8

Pages 1, 3, 5 and 7 were available from the start and the engine's tests were built on them (the rate equations, Eqs. 1–11, and the two worked examples). The even-numbered pages add the numerical comparisons, which are checked here. The initial score distribution is not stated for the example on page 6; $\mathbf{w}_{\text{init}} = (0.5, 0.5)$ reproduces the notes' numbers (other choices change the last digit or two of the differences, e.g. $\mathbf{w}_{\text{init}} = (1)$ gives 1.14×10^{-5} and 7.61×10^{-5}).

4.1 Defectors against Discriminators (pages 5–6)

Settings: `CN:00000-00 NC:00000000`, $\epsilon = 0.1$, $\epsilon_\uparrow = 0.4$, $\epsilon_\downarrow = 0.6$, $\rho_1 = \rho_2 = 0.5$.

Quantity	Notes	Closed form	Engine
$\pi_0^{(2)}$ (Discriminators)	$1/3$	0.333333	0.33333333
$\pi_0^{(1)}$ (Defectors)	0.754095	0.754095	0.75409483
$q(\boldsymbol{\pi}_0)$	0.456286	0.456286	0.456285920
Mean score, Defectors	0.326093	0.326093	0.32609316
Mean score, Discriminators	2	2	1.9999999988
Largest difference, scores 0–20, Defectors, 2000 steps	1.10×10^{-5}		1.097×10^{-5}
Largest difference, scores 0–20, Discriminators, 2000 steps	7.36×10^{-5}		7.361×10^{-5}

The closed form solves $0.1125y^2 + 0.135y - 0.04 = 0$ and uses $\pi_j^{(i)} = (1 - r_i)r_i^j$ with $r_1 = 0.04/(0.225\,q + 0.06)$ and $r_2 = 2/3$. The engine column is the converged distribution (tolerance 10^{-13}, 8129 steps) except for the last two rows, which are the 2000-step distribution the notes compare with. Figure 1 shows both distributions and the differences.

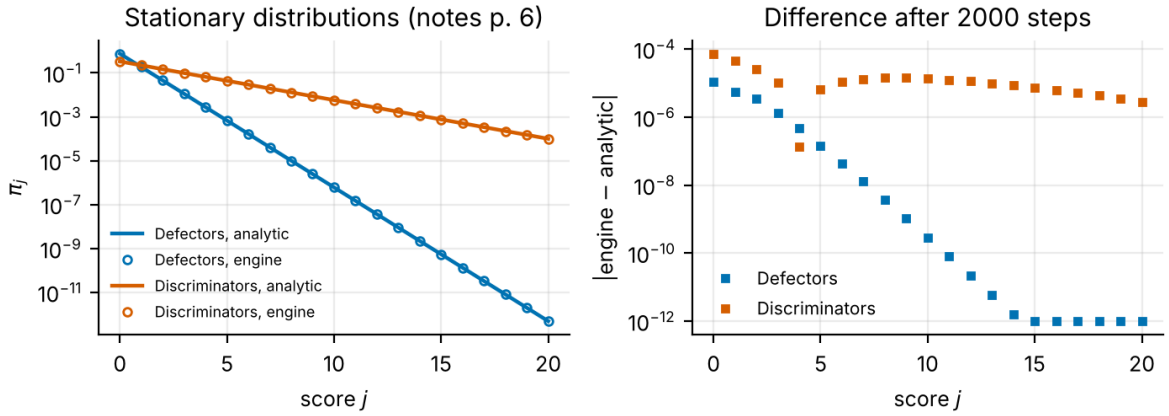


Figure 1: Defectors and Discriminators (notes, page 6). Left: closed-form stationary probabilities (lines) and the engine after 2000 steps from $(0.5, 0.5)$ (circles), scores 0 to 20, log scale. Right: absolute differences; values below 10^{-12} are drawn at 10^{-12}.

4.2 ϕ-Disc (pages 7–8)

Settings: `CN:00000-0+ NC:00000000`, a population made only of ϕ-Disc players.

Rates. The notes give $G_1 = 0$, $G_j = \dfrac{1 - w_0}{4(j-1)^\phi}$ for $j > 1$, $F_j = \dfrac{1 - w_0}{4}\left(1 - \dfrac{1}{j^\phi}\right)$ for $j \geq 1$, and $g_1 = \epsilon\epsilon_\uparrow$. The engine's g_j and f_j were compared with these (and Eqs. 3–4) for $\phi \in \{0, 0.17, 0.5, 1, 2\}$ on a score distribution with support on scores 0 to 6: the largest difference is 5.6×10^{-17}. For $\phi = 0$ the rates equal those of the Discriminators exactly (difference 0), as the notes say.

The case $\phi = 1$. With $\epsilon = 0.1$, $\epsilon_\uparrow = 0.4$, $\epsilon_\downarrow = 0.6$, the notes solve the normalisation equation for π_0 and give

$$\pi_k = \pi_0 \frac{\tilde{\epsilon}_\uparrow}{\tilde{\epsilon}_\downarrow} k\, r^{k-1} \frac{\Gamma(k+a)\,\Gamma(1+b)}{\Gamma(1+a)\,\Gamma(k+b)}, \qquad a = \frac{B}{\tilde{\epsilon}_\uparrow},\ b = \frac{\tilde{\epsilon}_\downarrow}{B + \tilde{\epsilon}_\downarrow},\ r = \frac{\tilde{\epsilon}_\uparrow}{B + \tilde{\epsilon}_\downarrow},$$

with $B = (1-\epsilon)(1-\pi_0)/4$, $\tilde{\epsilon}_\uparrow = \epsilon\epsilon_\uparrow$ and $\tilde{\epsilon}_\downarrow = \epsilon\epsilon_\downarrow$.

Quantity	Notes	Gamma solution	Engine
π_0	0.2555	0.255472	0.25547163 (converged), 0.255472 (2000 steps)
$\mathbb{E}$[score]	1.889	1.8886	1.888568 (converged), 1.8886 (2000 steps)
Largest $\lvert\pi_k^{\text{engine}} - \pi_k^{\text{analytic}}\rvert$, all scores			2.8×10^{-13} (converged, tolerance 10^{-14}, 807 steps), 7.8×10^{-16} (2000 steps from score 0, 440 scores)

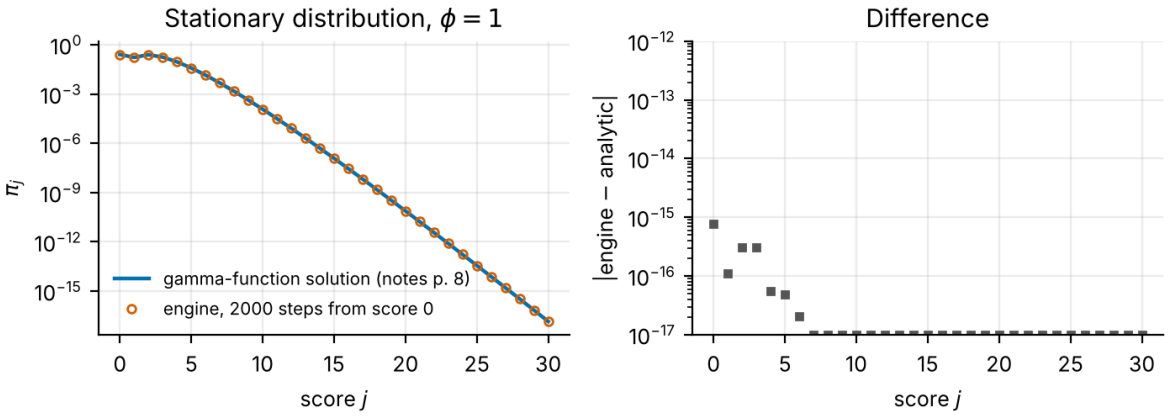


Figure 2: ϕ-Disc with $\phi = 1$ (notes, page 8). Left: the gamma-function solution (line) and the engine after 2000 steps from score 0 (circles), scores 0 to 30, log scale. Right: absolute differences; values below 10^{-17} are drawn at 10^{-17}.

Figure 2 shows the distribution and the differences.

4.3 The norm printed on page 7

The barcode lists the 8 entries in the order (needy player's class, whether the donor helped, donor's class). `CN:00000-00` has its minus sign at (positive, declined, positive): a donor with a positive score loses one when she does not help a needy player with a positive score, which is the norm of page 5. The ϕ-Disc norm adds a plus at (positive, helped, positive), giving `CN:00000-0+`. The printed `CN:000000-+` would put the minus at (positive, helped, score-0 donor), which both contradicts the sentence next to it and is not a legal norm (a score-0 player cannot lose a score; the engine rejects it).

5 Ternary classes and ϕ-Disc beyond the notes

5.1 Checks that were already in the suite

- *Lifted binary norms and strategies.* A ternary norm and ternary strategies that ignore the split at τ must give exactly the binary results. This is tested for $\tau \in \{1, 2, 5\}$ on a

set of binary norms (`test_ternary_that_ignores_tau_matches_binary`), and an ESS sweep of the binary CN-only norms lifted to ternary agreed with the binary sweep, with no differences.

- *A ternary strategy does use τ:* a strategy that treats scores $1..\tau$ differently from scores above τ gives a different answer when τ moves (`test_ternary_strategy_uses_tau`).
- *Probabilities.* Rates and payoffs are linear in each help probability; probability is conserved by every step; the stationary distribution satisfies detailed balance (Eqs. 5–6); chains whose up-rate at the top score is at least the down-rate are flagged as not converged.

5.2 Independent verification

Three independent checks were made after the engine was finished. The people (or agents) writing them did not use the engine's code to build their expected values.

A blind re-implementation. A second implementation (`tests/independent_model.py`, 137 lines) was written from the notes and `helper_funcs.jl` before the engine's source was opened. It computes G, F, g and f by direct enumeration (loops over the other player's strategy and score, the roles, and whether help is given) and iterates the chains itself. It reproduces the Julia outputs on 345 binary cases to 1.8×10^{-15} and the numbers of pages 4, 6, 7 and 8. Against the engine:

- 5000 random configurations of rates (binary, ternary and lifted norms; $\tau \in \{1, 2, 3, 5, 8\}$; one to three strategies of mixed kinds including probabilistic tables and ϕ-Disc with $\phi \in [0, 3]$; $\epsilon \in \{0, 0.1, 0.5, 1\}$; shares of zero; mass beyond τ and mass only in the top class): largest difference 2.2×10^{-16};
- 1780 chains of 25 to 400 steps: 1.1×10^{-15}, with mass conserved;
- stationary distributions: the independent fixed point of Eqs. 5–6 is unchanged by one engine step to 5×10^{-15} on 227 configurations, and the engine's own converged runs agree to 5×10^{-8} (set by its stopping rule);
- the three-segment closed form for a single class-based ternary strategy (100 cases), the ϕ-Disc rates of page 7 (1000 cases, 1.4×10^{-16}), the gamma-function solution of page 8 for six noise settings (2.8×10^{-16}), $\phi = 0$ against the Discriminators (identical), and strategies whose classes differ from the norm's.

An agent-based simulation. A population of 10^6 agents was simulated straight from the verbal model: each step every agent meets a random partner, roles are drawn, the Can helps with the strategy's probability, scores move by the norm, and with probability ϵ the move is replaced by noise. The score histograms were compared with the engine after the same number of steps, bin by bin with binomial standard errors. In 470 comparisons (the two example runs, ϕ-Disc alone, Defectors against Discriminators, random ternary norms with probabilistic strategies, a mix of ϕ-Disc and a ternary strategy) 468 were within five standard errors, and the total variation distances were at the level of pure sampling noise; the deviation falls as $N^{-1/2}$. The same yardsticks detect a 10% change in ϵ, a 4% change in the role factor, and noise applied only when an interaction happens. The two exceptions are a random case that the engine itself flags as drifting, where the default stopping tolerance leaves the engine short of the stationary law (see below).

Hand-computed cases and planted bugs. Eighteen small cases with $\tau = 1$ and 2 and a few occupied scores were worked out on paper from the notes (CN against NC, the recipient's

index against the donor's, the class boundary at score τ and $\tau+1$, the noise arithmetic, and the exponent of ϕ-Disc being the donor's score minus one for G_j); the engine matches them to 5.6×10^{-17}. Bugs were then planted in copies of the engine (a wrong cut-off, the wrong class for the donor, a transposed table, a wrong lift of a binary strategy to ternary, ϕ-Disc helping a recipient without a score) and each suite was run against them. The existing engine tests missed three cut-off bugs (the engine tests alone do not guard the boundary at τ); the independent tests and the hand cases catch all of them, and the engine tests catch the lift and transposition bugs that the simulation tests miss. All three suites are now in the repository (`test_independent_model.py`, 19 tests; `test_agent_based.py`, 12 tests at 10^5 agents; `test_hand_cases.py`), so each covers the gaps of the others.

What this found. No error in the rates, the norms, the strategies or the chains. Four points came up:

- At the default tolerance (10^{-5} on the change in one step) the engine stops before the stationary law for slowly relaxing chains, and the Julia code does the same, so results quoted at the default tolerance are approximate; the simulation sides with the tightly converged distribution.
- `Noise` accepted $\epsilon_\uparrow + \epsilon_\downarrow > 1$ and then produced negative "probabilities". It now refuses a sum above 1 (a sum below 1 is allowed; the notes take it to be 1).
- The README said a chain is drifting when the up-rate at the top score is at least the down-rate. When both are zero nothing can move and the engine, sensibly, does not call it drifting; the wording is corrected. Also, drifting does not strictly mean there is no stable distribution: in one configuration a stable distribution exists (it was found in closed form and is unchanged by an engine step) but the engine stayed in a slow drifting state for more than 3×10^4 steps.
- The two verifiers share the verbal model with the engine, so a misreading of the notes common to all of them would not be seen. The binary case is anchored to the Julia code. For ternary strategies and norms the conventions (the cut-off at τ, the order of the 18 characters, the orientation of the strategy table) rest on the description given for the engine and on the lifting identity, not on any independent reference.

6 Replicator dynamics, rest points and the GUI

There is no external reference for the replicator dynamics, so they are tested against closed forms and against games whose answers are known.

- `tests/test_replicator.py` (7 tests): the payoff matrix reproduces payoffs; Cooperators against Defectors follows the analytic solution; restarting the chains each step matches warm-starting when the scores settle; slow and fast time scales approach the quasi-static limit; the restart keeps the score range bounded; extinct strategies stay extinct and rest points stop the run; the role factor scales the interaction rates.
- `tests/test_simplex.py` (19 tests): the simplex grid; rest points and their stability for the prisoner's dilemma, hawk–dove, two- and three-strategy coordination games and rock–paper–scissors (stable and unstable versions); rest points of the engine's own payoffs against the invasion fitness; the awkward cases (an interior rest point close to an edge, segments of rest points, a zero eigenvalue, an eigenvalue close to the boundary, an edge point next to a vertex); parallel evaluation against serial.

- `tests/test_gui_logic.py` (47 tests): parsing and validation of every input, building strategies and norms, settings files, presets, and which strategies are shown at a given point of the simplex.
- Two browser tests (Chromium through Playwright, run with `-m e2e`) click through all three pages: choose presets, type a ternary strategy, fill in shares, scores, norm and noise, run, click the simplex, enter a point on the boundary (the absent strategy leaves the table, histogram and matrices), play the trajectory and download the data. A second test loads a settings file for two strategies. These were last run before the migration commits, which do not touch the GUI.

Reviews during development found and fixed: the order of the ternary strategy grid, a broken state when toggling between binary and ternary norms, missing validation of settings files, infinite or NaN inputs, a cap on the initial score, the name of a preset, thread-safety of the rate caches, round-off giving slightly negative rates, and several rest-point cases.

7 The earlier scripts on the engine

The old code in `src/` is deleted; the scripts that used it now run on the engine. For each, the old script was run on the same inputs as the new one, and the new one was compared with the results already committed in the repository. One difference is deliberate: the old code capped scores at `maxw` (mass at the cap could only move down), and the engine has no cap. Where the cap did not bind the two agree to rounding; where it did, the rates and class masses still agree in the committed runs while the score distributions now continue above `maxw`. `-maxw` is rejected.

Script	Comparison
Markov chain runner	`scripts/run_markov_chain.py` (was `run_markov_chain.py`). 20 argument sets (binary and three-state, one and two strategies, drifting and stable chains, three with a small `maxw`). Without a binding cap, rates differ by at most 3.6×10^{-16}, class masses by 1.4×10^{-15}, score distributions by 1.1×10^{-15}, and step counts are identical. 1800 random norms, strategies, shares and initial distributions: every step agrees to 1.6×10^{-14}. The seven committed run directories agree to 1.4×10^{-14} once scores from 100 up are added together. With a small `maxw` (10 to 15) the old cap changes the results: in one three-state case the rates differ by up to 0.012 and the class masses by up to 0.048, and in a binary case the old chain stopped after 132 steps because everything piled up at the cap, while the engine runs on and flags the chain as drifting.
Parallel ESS sweep	`experiments/ESS/find_ESS_parellel.py`. All 1296 CN-only norms: the old and new ESS lists are identical, the old code reproduces the committed CSV (333 rows, differences $\leq 1.1 \times 10^{-16}$), the port reproduces it to 10^{-15}, and all agree with Julia on 1296 of 1296 norms. The ternary path agrees with the old code in 64 of 64 cases (differences $\leq 10^{-15}$). The old code computed two-strategy payoffs from the distribution before the last step and the engine, like Julia, uses the final one; this reorders only near-ties (the largest gap is 2×10^{-9}) and changes no ESS.

Script	Comparison
Zeroness experiments	`experiments/verify-zeroness-one-strategy/`. All 20736 chains (1296 norms, 16 strategies): the same group of initial rates for every chain; initial rates identical, final rates within 1.1×10^{-15}, final share of score 0 within 4.3×10^{-15}; the number of tracked scores is the same in 10944 chains and larger in the 9792 chains where the old cap bound. The limit-check script (which keeps its own chain capped at `maxw`, since its claims are about that cap) reproduces the committed 20736-row table within 1.1×10^{-13} and the condition summary within 9×10^{-16}; its rates agree with the old rate function to 1.4×10^{-17} over all chains at 11 class masses.

8 Portability

The ESS sweep is meant for Rāpoi, whose Python module is 3.9.5. The engine and the sweep were run on Python 3.9 with numpy 1.26 and scipy 1.11 (517 engine tests and the sweep tests pass there). The full suite, including the GUI logic, passes on Python 3.13.16 with numpy 2.5.3, scipy 1.18.1, dash 4.4.1 and plotly 7.1.0.

9 Test inventory

`python -m pytest` runs 650 tests, all passing; two browser tests are deselected by default.

File	Tests	Covers
`test_against_julia.py`	460	400 stable-score cases, 60 sampled ESS rows
`test_gui_logic.py`	54	GUI parsing, validation, settings, presets
`test_engine.py`	32	norms, strategies, closed forms, ternary, invariants
`test_simplex.py`	19	grid, rest points, stability
`test_replicator.py`	7	dynamics, restart, role factor
`test_ess_sweep.py`	6	sweep: Slurm tasks, resume, merge
`test_experiments.py`	4	ESS, zeroness scripts vs results
`test_run_markov_chain.py`	4	saved rates drive the saved score histories
`test_known_norms.py`	12	Leading Eight structure and cooperative ESS
`test_plots.py`	20	ESS figure and its readers
`test_independent_model.py`	19	independent re-implementation, closed forms
`test_agent_based.py`	12	agent-based simulation
`test_hand_cases.py`	1	18 hand-computed cases
Total	650	

10 What has not been verified

- The notes have no ternary result and give a closed form for ϕ-Disc only at $\phi = 1$. For other ϕ the engine agrees with the rates in the notes, but no stationary distribution has been compared with a closed form.
- The role factor $q(1-q)$ is tested against its definition and by scaling, but Julia only ever uses $q = 1/2$.
- The replicator dynamics, the rest-point classification and the GUI have no reference implementation; they rest on the closed forms and games above.
- Chains with no stable distribution (for example the money norm with $\epsilon_\uparrow = \epsilon_\downarrow$) are reported after a fixed number of steps, so their values depend on the initial distribution and the number of steps.
- The Slurm scripts in `scripts/raapoi/` have been tested locally, with the Slurm environment variables set by hand, but have not been run on Rāpoi itself.

11 Reproducing the checks

```
python -m pytest                              # 650 tests
python -m pytest -m e2e                       # browser tests (needs playwright)
python tests/reference/compare_ess.py         # every ESS row against Julia
PYTHONPATH=. python docs/verification/make_figures.py   # the two figures
```